Block Wiedemann algorithm for fast color code decoding

Bohan Lu, Ben Greene

September 19, 2025

Abstract

In this project, we implement the block Wiedemann algorithm for solving sparse linear finite field equations efficiently.

1 Background and Motivation

Quantum error correction (QEC) is a central ingredient for building fault-tolerant quantum computers. At its heart lies the *decoder*: an algorithm that interprets noisy stabilizer measurement outcomes (syndromes) and determines a correction that protects the encoded quantum information. The performance of a decoder is measured not only by its accuracy in suppressing logical errors but also by its efficiency, as decoding must be performed repeatedly and at hardware timescales.

1.1 Color Codes and Their Appeal

Among the family of topological stabilizer codes, *color codes* occupy a special role. Defined on trivalent lattices such as the 6.6.6 hexagonal tiling, they admit transversal implementation of the full single-qubit Clifford group, making them attractive for architectures where gate locality and fault-tolerant logical operations are critical. In planar realizations with open boundaries, triangular color codes encode a single logical qubit with $n = m + 1$ physical qubits and m stabilizer checks.

Decoding color codes, however, has historically been more challenging than decoding surface codes. Their Tanner graphs contain many short cycles, which interfere with the convergence of traditional belief-propagation decoders, and the richer stabilizer structure increases the complexity of algorithm design.

1.2 Belief Propagation and Recent Advances

Belief propagation (BP) is a widely used heuristic for decoding quantum low-density parity-check (LDPC) codes. In BP, messages are iteratively passed along the Tanner graph between variable nodes (qubits) and check nodes (stabilizers), updating probabilities of error consistency. While conceptually appealing, straightforward BP often struggles with degeneracies and cycles in color-code graphs.

Recent work has revived BP-based decoding for color codes through clever scheduling and ensemble strategies. The VibeLSD approach (Koutsioumpas et al., 2025) runs multiple BP decoders in parallel, each with a different *serial* message-passing schedule, thereby breaking graph symmetries. The outputs of these decoders are then refined by a localized statistics decoder (LSD) post-processing stage, which enforces consistency with the syndrome and resolves residual ambiguities. This combination achieves record-breaking performance, placing color-code thresholds and footprints on par with those of the surface code.

1.3 Our Algebraic Approach: Minimal Polynomials over GF(4)

While BP-style methods rely on probabilistic message passing, our approach is inherently algebraic. The parity-check matrix of a color code can be expressed over the finite field GF(4), with entries encoding the X/Z action of stabilizers. For triangular color codes encoding one logical qubit, the check matrix has dimensions $m \times (m+1)$. Removing any single column produces an $m \times m$ square submatrix, yielding n such submatrices in total.

A key observation is that each submatrix admits a *minimal polynomial* $m_A(x)$ over GF(4), defined as the monic polynomial of least degree that annihilates the matrix. Crucially, the repertoire of minimal polynomials over GF(4) is finite and independent of n. This means that rather than performing costly $O(m^3)$ linear-algebraic computations per submatrix, we can reduce decoding to a lightweight *classification task*: identify which minimal polynomial arises and consult a small lookup table.

1.4 Complexity and Motivation

The implication is profound. By mapping each of the n possible submatrices to one of a finite set of minimal polynomials, the decoding process scales linearly in the number of qubits,

$$O(n),$$

instead of cubic scaling per submatrix. This transforms the problem of decoding single-logical color codes into one of efficient spectral recognition, offering both speed and determinism. Our goal is to benchmark this algebraic decoder against state-of-the-art BP-based methods such as VibeLSD, under realistic circuit-level noise and syndrome extraction settings, to determine whether such minimal-polynomial classification can provide a competitive or superior alternative.

1.5 BP-Guided Selection of the Square Submatrix

Our algebraic decoder uses belief propagation not as the final decision rule, but as a *selector* that identifies which column deletion is most promising to analyze. Recall that a triangular colour code with one logical qubit has a check matrix $H_4 \in \mathrm{GF}(4)^{m\times(m+1)}$; removing any column j yields a square submatrix $A_j \in \mathrm{GF}(4)^{m\times m}$. Instead of scanning all $n = m+1$ possibilities, we run a lightweight BP pass to obtain soft marginals over qubits and rank columns by a syndrome-tension score (e.g., the sum of residual inconsistencies at neighbouring checks under the local-most-likely error).

Concretely:

1. Run serial-schedule BP for a small, fixed number of iterations (e.g., 5–10) to produce variable-node beliefs $\{b_j\}$.
2. Score each column j (qubit) by a monotone function of b_j and its adjacent checks ("syndrome tension").
3. Select the top-k candidates (typically $k \in \{1, 3, 5\}$).
4. For each candidate j, form A_j and compute its minimal polynomial $m_{A_j}(x)$ via the Wiedemann+Berlekamp–Massey pipeline (matvec-only). Classify A_j by a finite lookup of GF(4) minimal polynomials and choose the correction consistent with the measured syndrome.

This *BP-as-selector* reduces constant factors by avoiding a full scan over all n submatrices while preserving the algebraic decoder's determinism and low latency once the minimal-polynomial repertoire is cached. The BP stage costs $O(E \cdot T)$ for T iterations on a Tanner graph with E edges; the algebraic stage costs $O(k)$ lookups plus a small number of matvecs per candidate.

2 Benchmark Plan

Having motivated both the belief-propagation ensemble approach (VibeLSD) and our algebraic minimal-polynomial decoder, the next step is to design a fair and systematic comparison. Our guiding principle is a like-for-like evaluation: the two decoders should be tested under identical circuit constructions, noise models, and code distances, with results reported in a common set of metrics. We evaluate our hybrid decoder in two stages: (i) a BP-guided submatrix selector with a fixed, small iteration budget; and (ii) an algebraic classifier based on minimal polynomials over GF(4) computed via Wiedemann+BM.

2.1 Codes, Circuits, and Noise Models

We restrict attention to planar 2D color codes that have been the focus of recent benchmarks:

- **Codes:** The hexagonal 6.6.6 code as our primary target, supplemented by the square-octagon 4.8.8 code as a secondary check of generality.
- **Distances:** $d \in \{3, 5, 7, 9, 11, 13\}$, covering both small-scale and moderate-distance regimes accessible in simulation.
- **Circuits:** Bell-flagged, Tri-optimal, Superdense, and Midout syndrome extraction circuits, implemented with the same ancilla scheduling as in prior work.
- **Noise models:** Circuit-level depolarizing noise ("Uniform") and the superconducting-inspired Si1000 model, with parameters matched to published tables. Where available, we also consider biased and leakage variants.

2.2 Measured Quantities

For each decoder and configuration we will record:

1. **Logical error rate per round** p_L as a function of physical error rate p, plotted on log–log axes.
2. **Pseudo-thresholds** and, where possible, finite-size threshold estimates from crossing analysis.
3. **Footprint:** the number of physical qubits (including ancillas and flags) required to reach target logical error rates (e.g. 10^{-3}, 10^{-5}).
4. **Decoder latency and throughput:** per-round wall time and syndromes/sec, measured on a single CPU core, a many-core CPU, and a GPU. Both median and tail latencies (95th/99th percentile) will be reported.
5. **Parallelism profile:** effective parallel fraction (Amdahl's law) and hardware occupancy. For VibeLSD this includes ensemble size and iteration cap; for our decoder it includes matvec costs and Berlekamp–Massey sequence length.
6. **Validity rate:** the fraction of instances where the decoder returns a syndrome-consistent correction without fallback. For VibeLSD this corresponds to residual edits required in the LSD stage; for our decoder it corresponds to reclassification steps in the minimal-polynomial lookup.

2.3 Reproducible Simulation Protocol

To ensure comparability and reproducibility, we will:

1. Implement each syndrome extraction circuit with identical gate orderings and verify stabilizer commutation and hook suppression.
2. Validate noise model implementations against published benchmarks by reproducing selected datapoints.
3. Simulate $N = 10^5$–10^6 rounds per parameter point, depending on target confidence intervals, using common random seeds for both decoders.
4. Log raw results (syndrome, correction, success/failure, timing) in a standardized format (e.g. parquet) with metadata such as commit hash, seed, and hardware.
5. Aggregate into logical error estimates with confidence intervals, latency quantiles, and footprint curves.

2.4 Success Criteria and Trade-offs

We will consider our minimal-polynomial decoder competitive if it achieves:

- Logical error rates p_L that are not worse than VibeLSD within confidence intervals.
- Equal or smaller footprint at practical logical error targets $(10^{-3}, 10^{-5})$.
- Comparable or lower median and tail latencies at matched hardware.
- Robustness to seed variation and mild model mismatch.

We also note the qualitative trade-offs: our algebraic approach promises deterministic, low-latency decoding once the minimal-polynomial repertoire is cached, while VibeLSD leverages probabilistic diversity and a strong post-processor at the cost of ensemble overhead. The benchmark plan is designed to surface these contrasts clearly.

3 Minimal polynomials and Wiedemann

3.1 Scalar Wiedemann Algorithm

Solving sparse linear systems over finite fields is a central problem in computational algebra, coding theory, and quantum error correction. Traditional dense methods like Gaussian elimination are computationally infeasible for large-scale problems due to their $\mathcal{O}(n^3)$ complexity and memory demands. As an alternative, the Wiedemann algorithm [Wie86] offers a randomized, projection-based method for solving such systems over finite fields, particularly GF(2) * Gby leveraging Krylov subspace techniques and minimal polynomial extraction.

Given a sparse matrix $A \in \mathrm{GF}(2)^{n\times n}$ and vector $b \in \mathrm{GF}(2)^n$, the scalar Wiedemann algorithm generates a sequence of scalar values:

$$s_t = u^\top A^t b, \quad t = 0, 1, \ldots, 2n-1$$

using a randomly chosen projection vector $u \in \mathrm{GF}(2)^n$. This sequence is governed by a linear recurrence, and its minimal polynomial $f(x)$ is computed using the Berlekamp–Massey algorithm [Ber15].

From $f(x)$ a linear combination of Krylov vectors $\{A^i b\}$ is used to recover a solution x satisfying $Ax = b$. The method is probabilistic but effective when multiple projections are used or in moderately sized systems.

3.2 Berlekamp–Massey Algorithm over GF(2)

Originally designed for decoding BCH codes [Ber15], the Berlekamp-Massey algorithm finds the shortest linear feedback shift register (LFSR) that generates a given sequence over a field. For sequences from scalar Wiedemann, it identifies a polynomial $f(x)$ such that:

$$s_t + c_1 s_{t-1} + \cdots + c_d s_{t-d} = 0, \quad \forall t \geq d$$

Our implementation uses bitwise operations (XOR, AND) to reflect GF(2) arithmetic, optimized for eventual GPU acceleration.

3.3 Block Wiedemann Algorithm

The scalar approach becomes unreliable on large or degenerate matrices due to poor projection coverage. The *Block Wiedemann algorithm*, introduced by Coppersmith [**?**], generalizes the method by projecting into multiple random subspaces:

$$S_t = U^\top A^t V, \quad U, V \in \mathrm{GF}(2)^{n\times b}$$

where $S_t \in \mathrm{GF}(2)^{b\times b}$ is a sequence of small matrices satisfying a matrix recurrence. This recurrence can be recovered using a block variant of Berlekamp–Massey [**?**] or the Mulders–Storjohann algorithm.

The block version improves robustness and enables parallelization—essential for high-throughput applications such as quantum decoder backends and symbolic algebra on GPUs.

3.4 Project Roadmap

This implementation proceeds in four stages, each aligned with weekly development milestones:

1. Implement scalar Wiedemann and Berlekamp–Massey over GF(2), including unit tests and verification
2. Build the Block Krylov sequence engine and sparse matrix operations
3. Implement and validate a block Berlekamp–Massey solver for matrix recurrence
4. Benchmark performance on large-scale sparse systems and prepare for GF(4) and CUDA backends

Ultimately, this solver forms the foundation for future GF(4) decoding engines and symbolic algebra toolchains, integrated with GPU-accelerated linear algebra frameworks.

3.5 Wiedemann method (matrix → scalar recurrence)

- **Wiedemann's trick** is to reduce the matrix problem into a **scalar linear recurrence problem**.
- You pick random test vectors:
  - $u \in \mathrm{GF}(q)^n$ (left vector),
  - $v \in \mathrm{GF}(q)^n$ (right vector).
- Form the sequence
$$s_t = u^\top A^t v, \quad t = 0, 1, \ldots, 2n-1.$$
- This scalar sequence **must satisfy** the same recurrence as the minimal polynomial of A.
- Then you just feed $s_0, \ldots, s_{2n-1}$ into **Berlekamp–Massey**.

So the "Wiedemann" part is exactly the sequence-building loop:

```
seq = Field.Zeros(2*n)
power_vec = right_vec.copy()
for t in range(2*n):
    seq[t] = left_vec @ power_vec   # scalar = u^T (A^t v)
    power_vec = matrix @ power_vec  # update A^t v -> A^(t+1) v
```

That's Wiedemann.

3.6 Berlekamp–Massey (sequence → minimal polynomial)

- Given a sequence over GF(4), BM finds the shortest linear recurrence it satisfies.
- That recurrence polynomial is a **candidate minimal polynomial**.

3.7 Whole thing together

So the function:

```
def minimal_polynomial_wiedemann(matrix, tries=2):
    ...
    for _ in range(tries):
        left_vec = Field.Random(n)
        right_vec = Field.Random(n)

        # --- THIS is the Wiedemann step ---
        seq = Field.Zeros(2*n)
        power_vec = right_vec.copy()
        for t in range(2*n):
```

```
            seq[t] = left_vec @ power_vec
            power_vec = matrix @ power_vec
        # --- END of Wiedemann; now BM on seq ---

        candidate_poly = berlekamp_massey(seq)
        minpoly = galois.lcm(minpoly, candidate_poly)
    return minpoly
```

So:

- **Wiedemann = how we produce the sequence.**
- **BM = how we decode that sequence into a polynomial.**
- **LCM = combine across trials** until you hit the true minimal polynomial of the whole matrix.

References

[Ber15] Elwyn R. Berlekamp. *Algebraic Coding Theory*. World scientific, Hackensack (N. J.), rev. ed edition, 2015.

[Wie86] D. Wiedemann. Solving sparse linear equations over finite fields. *IEEE Trans. Inform. Theory*, 32(1):54–62, January 1986.